Table des matières

Exercices corrigés 12 — SQL analytique appliqué 1
Questions courtes 1
Questions pratiques 2
Diagnostics 3
Questions à développer 4
Mises en situation 5
Corrigé 6

Exercices corrigés 12 — SQL analytique appliqué

Comment travailler ces exercices. Ce module est un projet, et ses exercices lui ressemblent : ils demandent moins d'écrire du SQL que de défendre ce que vous avez écrit. Les questions courtes balaient les douze modules ; les pratiques vous font concevoir plutôt que taper ; les diagnostics vous montrent des livrables plausibles et faux ; les questions à développer et les mises en situation sont l'exercice d'un entretien technique. Comptez trois heures et demie pour y répondre, le corrigé non compris.

Les repères de terrain. Ce module n'a qu'un laboratoire, sur l'entrepôt. Un énoncé marqué `[OLAP]` se traite donc dans DuckDB sur `adventureworksdw.duckdb`, ou sur le schéma `RAPPORT` que vous avez construit. Les énoncés sans marque ne demandent pas de moteur : ils demandent un raisonnement.

Le corrigé suit les énoncés, en fin de document. Répondez à l'énoncé en entier avant d'y passer, et surtout répondez à voix haute : la moitié de ces questions se posent en entretien, et savoir les écrire ne suffit pas.

Questions courtes

1. Citez les cinq temps de la méthode, dans l'ordre, et dites ce qu'on perd à en inverser deux.
2. La demande de départ contenait trois mots indéfinis. Nommez-les, et donnez pour chacun deux interprétations qui mènent à deux rapports différents.
3. Qu'est-ce que le grain d'une table de faits, et pourquoi celui de `dbo.FactInternetSales` surprend- il ?
4. Vous voulez le nombre de commandes à partir d'une requête au grain article. Qu'écrivez-vous, et qu'arrive-t-il si vous l'oubliez ?
5. Vous sommez le montant total d'une commande dans une requête au grain article. Que vaut le résultat, et de combien est-il faux en moyenne sur ce jeu de données ?

© 2026 iAlexMG. Tous droits réservés.

6. Pourquoi la période du rapport a-t-elle été fixée du 1er janvier 2011 au 31 décembre 2013, alors que la donnée va plus loin des deux côtés ?
7. Que contient une note de cadrage, et pourquoi est-elle le livrable qui vaut le plus six mois plus tard ?
8. Pourquoi ne peut-on pas joindre le fait à toute la dimension de date ?
9. À quoi sert la colonne `jour_reconstitue`, et quelle question permet-elle de trancher plus tard ?
10. Sur l'axe complet de ce projet, le cadre en lignes et le cadre en durée donnent le même résultat. Pourquoi, et qu'est-ce que cela vous autorise à écrire ?
11. Pourquoi le classement des produits groupe-t-il sur `ProductAlternateKey` plutôt que sur `ProductKey` ?
12. Les jointures vers la sous-catégorie et la catégorie sont des `LEFT JOIN`. Que se passerait-il avec des jointures internes, et comment le sauriez-vous ?
13. Un contrôle de qualité qui ne trouve rien est-il inutile ? Justifiez, puis dites ce que vous en faites dans le livrable.
14. Trois des contrôles de la partie 4 trouvent quelque chose. Lesquels sont des anomalies et lequel ne l'est pas ?
15. L'as-of join de la partie 5 n'emploie jamais `EndDate`. Pourquoi, sur cette dimension en particulier ?
16. Le fait porte déjà `TotalProductCost`. Donnez la raison pour laquelle on recalcule quand même la marge.
17. À quoi servent les colonnes issues de `GROUPING()` dans le rapport de synthèse ?
18. Pourquoi livrer les mesures au format long plutôt qu'en tableau croisé, alors que le lecteur veut un tableau croisé ?
1. Dans quel ordre optimise-t-on et vérifie-t-on l'exactitude ? Justifiez en une phrase.
2. Un rafraîchissement en deux commandes, suppression puis insertion, sans transaction. Décrivez ce que voit un lecteur, et pourquoi personne ne le signalera.

Questions pratiques

Ces énoncés demandent de concevoir. Écrivez le SQL quand il est court, et l'esquisse quand il est long : ce qui est évalué est la décision, pas la frappe.

P1. `[OLAP]` Un collègue veut ajouter la mesure « panier moyen par territoire et par mois ». Dites à quel grain elle se calcule, écrivez la requête, et expliquez pourquoi la moyenne des paniers moyens quotidiens ne donne pas le même résultat.

P2. `[OLAP]` Ajoutez à `V_TENDANCE` une colonne qui compare chaque jour au même jour de l'an précédent. Traitez explicitement le cas du 29 février et celui du premier exercice, où il n'y a pas d'an précédent.

© 2026 iAlexMG. Tous droits réservés.

P3. [OLAP] Écrivez le contrôle qui vérifie qu'aucune commande n'a deux lignes portant le même numéro de ligne, et le contrôle qui vérifie qu'une commande appartient toujours à un seul client. Dites lequel des deux est le plus susceptible de trouver quelque chose sur une vraie source.

P4. [OLAP] Votre vue `V_LIGNE_ENRICHIE` renvoie des lignes sans version de produit valide. Proposez trois traitements possibles de ces lignes dans le rapport final, et dites lequel vous retenez et pourquoi.

P5. [OLAP] Concevez la table de journal du chargement : donnez ses colonnes, dites qui l'alimente, et écrivez la requête qu'un opérateur lance le matin pour savoir si tout s'est bien passé. (avancé)

P6. [OLAP] Le rapport doit maintenant couvrir aussi les ventes des revendeurs, qui vivent dans une autre table de faits, au même grain. Dites ce que vous changez dans chacune des huit parties du laboratoire, et ce qui ne change pas. (avancé)

Diagnostics

E1. Un collègue livre ce chiffre comme « nombre de commandes de 2013 » :

```sql
SELECT COUNT(*) FROM dbo.FactInternetSales WHERE YEAR(OrderDate) = 2013;
```

Deux défauts, de natures différentes. Nommez-les et corrigez.

E2. Une tendance mensuelle livrée telle quelle :

```sql
SELECT DATE_TRUNC('month', OrderDate) AS mois, SUM(SalesAmount) AS ventes
FROM   dbo.FactInternetSales
GROUP  BY 1 ORDER BY 1;
```

Le lecteur conclut que les ventes se sont effondrées en janvier 2014 et que décembre 2010 a été catastrophique. Expliquez, et dites ce que vous changez dans la requête et dans le livrable.

E3. Un classement des meilleurs produits :

```sql
SELECT p.EnglishProductName, SUM(f.SalesAmount) AS ventes
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimProduct AS p ON p.ProductKey = f.ProductKey
GROUP  BY 1 ORDER BY ventes DESC LIMIT 10;
```

Deux produits du catalogue apparaissent chacun deux fois dans le résultat avec des montants différents. Expliquez et corrigez.

E4. Un rapport de marge :

```sql
SELECT SUM(f.SalesAmount - v.StandardCost * f.OrderQuantity) AS marge
FROM   dbo.FactInternetSales AS f
```


© 2026 iAlexMG. Tous droits réservés.

```
JOIN   dbo.DimProduct AS ref ON ref.ProductKey = f.ProductKey
JOIN   dbo.DimProduct AS v   ON v.ProductAlternateKey = ref.ProductAlternateKey
WHERE  f.OrderDate BETWEEN v.StartDate AND v.EndDate;
```

Elle renvoie `NULL`. Donnez les deux causes et l'écriture correcte.

E5. Une synthèse avec sous-totaux, livrée dans un fichier :

```
SELECT COALESCE(categorie, 'Total') AS categorie,
       COALESCE(pays, 'Total')      AS pays,
       SUM(ventes)                  AS ventes
FROM   ventes_detail
GROUP  BY ROLLUP (categorie, pays);
```

Le destinataire additionne la colonne des ventes dans son tableur et obtient trois fois le chiffre d'affaires réel. Expliquez le facteur trois, et dites ce qui manque au livrable. (avancé)

E6. Un chargement quotidien censé être rejouable :

```
DELETE FROM rapport.fait WHERE jour = :jour;
INSERT INTO rapport.fait
SELECT * FROM source.fait WHERE jour >= :jour;
```

Il produit des doublons dès qu'on le relance. Trouvez l'erreur, et dites ce que le contrôle de la partie 4 aurait détecté. (avancé)

Questions à développer

A1. Le module affirme que le cadrage se fait avant le SQL et ne se rattrape pas après. Défendez cette affirmation en un paragraphe, puis décrivez ce qui arrive concrètement à un projet où le cadrage est fait après coup, en suivant une mesure précise du laboratoire.

A2. Les contrôles de la partie 4 précèdent les mesures de la partie 3 dans l'ordre logique, alors que le laboratoire les place après. Expliquez pourquoi cet ordre a été retenu, dites ce qu'il coûte, et donnez l'ordre que vous adopteriez sur un projet réel.

A3. La partie 5 propose trois traitements de l'écart entre la marge du fait et la marge recalculée. Exposez-les, dites lequel vous retenez, et surtout à quelles conditions vous changeriez d'avis. Traitez explicitement le cas où l'écart est important et inexplicable à la veille de la livraison.

A4. Un collègue soutient qu'un rapport juste suffit, et que la partie 8 est du travail d'administrateur qui ne concerne pas un analyste. Répondez-lui en un paragraphe, en reconnaissant ce que son objection a de fondé, puis en montrant par un scénario concret ce que son rapport deviendra en son absence.

A5. Le capstone ne remobilise pas le Cours 08. Expliquez pourquoi, dites où l'étape correspondante se placerait si la source en contenait, et ce qu'elle changerait aux parties 4 et 7. (avancé)

© 2026 iAlexMG. Tous droits réservés.

Mises en situation

M1. Coopérative Trois-Pistoles, la demande de la direction. La direction vous demande « un tableau de bord des ventes, comme celui de l'an dernier, mais en mieux ». Le tableau de bord de l'an dernier existe, personne ne sait comment il est calculé, son auteur est parti, et deux services citent des chiffres différents pour le même mois.

Décrivez votre démarche, dans l'ordre. Dites ce que vous faites du tableau existant, ce que vous demandez et à qui, et comment vous traitez la divergence entre les deux services sans transformer la réunion en tribunal. Précisez ce que vous livrez en premier, et pourquoi ce n'est probablement pas un tableau de bord.

M2. Manufacture Joliette, le chiffre qui a changé. Votre rapport tourne depuis trois mois. Ce matin, un directeur vous écrit que le chiffre d'affaires d'octobre, qu'il avait noté à 4,1 millions, en affiche maintenant 4,4. Personne n'a modifié votre code.

Listez toutes les causes possibles de ce changement, classées de la plus probable à la moins probable, et dites pour chacune comment vous la vérifiez. Décrivez ensuite ce que vous répondez au directeur avant d'avoir fini l'enquête. Précisez enfin ce que vous mettriez en place pour que la question ne se pose plus dans les mêmes termes la prochaine fois.

M3. Distributeur Verchères, l'entretien technique. Vous présentez ce projet en entretien. Le recruteur ouvre votre code, s'arrête sur la vue de l'as-of join, et vous dit : « pourquoi cette complexité, il y a déjà une colonne de coût dans le fait ».

Répondez-lui. Votre réponse doit tenir en deux minutes à l'oral, commencer par reconnaître ce que la question a de légitime, et se terminer par ce que vous avez mesuré. Le recruteur enchaîne : « et si je vous disais que sur mon entrepôt, cette écriture met quarante minutes ». Répondez encore.

M4. Assurance Baie-des-Sables, le rapport hérité. Vous héritez d'un rapport de 900 lignes de SQL, d'une seule requête, sans commentaire, qui produit le tableau de bord mensuel de la direction. Il fonctionne. Il met vingt minutes. Personne n'ose y toucher, et la direction demande deux mesures supplémentaires pour le trimestre prochain.

Décrivez votre plan, en étapes datées. Dites ce que vous faites en premier, et pourquoi ce n'est ni de le réécrire ni d'ajouter les deux mesures. Traitez explicitement la question de la non-régression : comment prouvez-vous que votre version donne les mêmes chiffres que l'ancienne, et que faites-vous si elle donne des chiffres différents et que les vôtres semblent plus justes ? (avancé)

© 2026 iAlexMG. Tous droits réservés.

Corrigé

La numérotation est celle des énoncés. Sur ce module plus que sur les autres, plusieurs réponses se défendent : le corrigé donne la réponse attendue, puis dit ce qui est également acceptable et pourquoi.

Questions courtes

1. Cadrer, contrôler, construire, mesurer, rendre rejouable. Inverser cadrer et construire donne un rapport qu'il faut refaire, parce que le grain et les définitions se choisissent avant d'écrire. Inverser mesurer et construire fait optimiser du code dont on ignore encore s'il est juste, donc travailler deux fois.

Théorie : `Theorie_12_SQL.md`, section 2.

2. « Ventes » peut être le montant facturé taxes comprises ou le montant de ligne hors taxes, et l'écart change tous les chiffres. « Tendance » peut être la variation d'un mois sur l'autre ou une moyenne mobile, et les deux ne racontent pas la même histoire sur une série irrégulière. « Meilleurs produits » peut être par montant ou par marge, et le classement n'est pas le même : un produit à gros volume et faible marge domine le premier et disparaît du second.

Théorie : `Theorie_12_SQL.md`, section 2.

3. Le grain est ce qu'une ligne de la table représente. Celui de `dbo.FactInternetSales` surprend parce qu'une ligne est un article d'une commande, pas une commande : 60 398 lignes pour 27 659 commandes.

Théorie : `Theorie_12_SQL.md`, section 3.

4. `COUNT(DISTINCT SalesOrderNumber)`. Sans lui, on compte des articles et on les appelle des commandes, ce qui donne un nombre plus de deux fois trop grand.

Théorie : `Theorie_12_SQL.md`, section 3.

5. Il vaut le montant de la commande multiplié par son nombre d'articles. Sur ce jeu de données, le facteur moyen est de 60 398 sur 27 659, soit environ 2,2. L'erreur est d'autant plus difficile à repérer qu'elle produit un nombre du bon ordre de grandeur.

Théorie : `Theorie_12_SQL.md`, section 3.

6. Parce que décembre 2010 ne compte que trois jours de vente et janvier 2014 s'arrête au 28. Les inclure ferait commencer et finir tout graphique mensuel par une falaise qui n'est pas un fait économique, et le lecteur en tirerait une conclusion fausse avant d'avoir lu la note.

Théorie : `Theorie_12_SQL.md`, section 3.

© 2026 iAlexMG. Tous droits réservés.

7. Le grain, la période et sa complétude, la définition de chaque mesure en une phrase validable par un non-technicien, et les décisions prises sur les cas limites. Elle vaut le plus six mois plus tard parce que le code dit comment on calcule et jamais pourquoi : un chiffre sans définition n'est pas un chiffre, c'est une source de discussions.

Théorie : `Theorie_12_SQL.md`, sections 2 et 5.

8. Parce que `dbo.DimDate` couvre 2005 à 2014, alors que les ventes commencent fin 2010. La série obtenue commencerait par six ans de zéros. Une dimension de date est faite pour être filtrée.

Théorie : `Theorie_12_SQL.md`, section 4.

9. À distinguer un zéro observé d'un zéro fabriqué par l'axe. Elle permet de trancher, plus tard, la question de savoir si la moyenne demandée porte sur la période ou sur les jours actifs, et de recalculer l'autre sans reconstruire la vue.

Théorie : `Theorie_12_SQL.md`, section 4.

10. Parce que l'axe est complet : une ligne égale exactement un jour, donc sept lignes égalent sept jours. Cela autorise à écrire `ROWS`, qui est portable, plutôt que le cadre en durée, dont le support varie d'un moteur à l'autre.

Théorie : `Theorie_12_SQL.md`, section 4.

11. Parce que `ProductKey` désigne une version du produit, pas le produit. Un produit qui a changé de version pendant la période compterait comme deux produits distincts, et chacune de ses deux moitiés se classerait plus bas que le produit entier ne le mérite.

Théorie : `Theorie_12_SQL.md`, section 4.

12. Les produits sans sous-catégorie disparaîtraient du classement, silencieusement. On le saurait en comparant le total du classement à la somme des ventes de la période : l'écart est la mesure exacte de ce qui a disparu. C'est le contrôle qu'il faut prendre l'habitude d'écrire après toute jointure de dimension.

Théorie : `Theorie_12_SQL.md`, section 4.

13. Non. Il vous apprend que le chiffre suivant est solide, ce qui est une information, et il vaut d'être conservé dans le livrable avec son résultat. Le jour où il trouvera quelque chose, vous saurez que la situation est nouvelle plutôt que d'avoir à vous demander si elle a toujours été comme ça.

Théorie : `Theorie_12_SQL.md`, sections 4 et 5.

14. Les 448 produits jamais vendus et le territoire sans vente en ligne ne sont pas des anomalies : un catalogue contient des produits qui ne se vendent pas en ligne, et le territoire restant est le siège social. Les 200 intervalles de validité incohérents en sont une, et c'est elle qui décide de la façon d'écrire la partie 5.

Théorie : `Theorie_12_SQL.md`, section 4.

© 2026 iAlexMG. Tous droits réservés.

15. Parce que les 200 dates de fin de cette dimension sont toutes antérieures à leur date de début, et que les 406 autres lignes n'ont pas de date de fin du tout. Une écriture qui lit `EndDate` ne peut donc apparier personne, et elle renvoie zéro ligne sans erreur.

Théorie : `Theorie_12_SQL.md`, section 4.

16. Parce que rien ne garantit que la colonne du fait porte le coût du moment plutôt que le coût au chargement, et qu'on a maintenant les moyens de le vérifier plutôt que de le croire. Le recalcul est un contrôle avant d'être une mesure.

Théorie : `Theorie_12_SQL.md`, section 4.

17. À distinguer une ligne de sous-total d'une ligne dont la valeur est réellement inconnue. Sans elles, le `COALESCE` qui met une étiquette lisible rend les deux cas indiscernables, et il aggrave la confusion au lieu de la lever.

Théorie : `Theorie_12_SQL.md`, section 4.

18. Parce qu'un rapport au format long accepte une mesure de plus sans changer de structure, alors qu'un tableau croisé figé dans une vue oblige à réécrire la vue à chaque ajout. La transposition est le travail de l'outil de restitution, et c'est là qu'elle coûte le moins cher.

Théorie : `Theorie_12_SQL.md`, sections 4 et 5.

19. On vérifie l'exactitude d'abord, on optimise ensuite. Optimiser un rapport dont on n'a pas vérifié l'exactitude fait travailler deux fois : la correction changera le plan, donc invalidera l'optimisation.

Théorie : `Theorie_12_SQL.md`, section 2.

20. Il voit la table vide, ou partiellement remplie, et il affiche zéro ou un total incomplet. Personne ne le signalera parce qu'aucune erreur n'est levée, que le chiffre a la bonne forme, et que le rapport suivant sera juste.

Théorie : `Theorie_12_SQL.md`, section 4.

Questions pratiques

P1. `[OLAP]` Le panier moyen se calcule au grain commande, puis s'agrège. La faute consiste à le calculer au grain article.

```
WITH par_commande AS (
    SELECT f.SalesOrderNumber,
           ANY_VALUE(f.SalesTerritoryKey)        AS territoire,
           DATE_TRUNC('month', MIN(f.OrderDate)) AS mois,
           SUM(f.SalesAmount)                    AS montant
    FROM   dbo.FactInternetSales AS f
    GROUP  BY f.SalesOrderNumber
)
```


© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT t.SalesTerritoryCountry AS pays, c.mois,
       COUNT(*)                AS commandes,
       AVG(c.montant)          AS panier_moyen
FROM   par_commande AS c
JOIN   dbo.DimSalesTerritory AS t ON t.SalesTerritoryKey = c.territoire
GROUP  BY 1, 2 ORDER BY 1, 2;
```

La moyenne des paniers moyens quotidiens ne donne pas le même résultat parce qu'elle pèse chaque jour également, quel que soit son nombre de commandes. Un mardi à trois commandes compte autant qu'un vendredi à trois cents. C'est une moyenne de moyennes, et elle répond à une autre question que celle posée.

Théorie : `Theorie_12_SQL.md`, section 3.

P2. [OLAP]

```sql
SELECT jour, ventes,
       LAG(ventes, 364) OVER (ORDER BY jour) AS meme_jour_an_dernier,
       ventes - LAG(ventes, 364) OVER (ORDER BY jour) AS ecart
FROM   RAPPORT.V_SERIE_JOUR;
```

Le décalage de 364 plutôt que 365 est le choix à défendre : il aligne le même jour de la semaine, ce qui est ce qu'on veut sur une série de ventes au détail, où le jour de la semaine explique une bonne part de la variation. Un décalage de 365 alignerait la date exacte et comparerait un mardi à un mercredi.

Le 29 février se règle de lui-même avec cette approche, puisqu'on décale d'un nombre de lignes et non d'une durée calendaire. Avec une jointure sur la date de l'an précédent, il faudrait décider explicitement à quoi comparer le 29 février, et le plus honnête est de laisser `NULL`.

Le premier exercice n'a pas d'an précédent, donc la colonne vaut `NULL` sur 364 lignes. Il faut le laisser à `NULL` et surtout ne pas mettre zéro : un écart de zéro se lit comme une stabilité, et une donnée absente se lit comme une donnée absente.

Théorie : `Theorie_12_SQL.md`, section 4.

P3. [OLAP]

```sql
-- Unicité de la ligne de commande.
SELECT COUNT(*) AS doublons
FROM   (SELECT SalesOrderNumber, SalesOrderLineNumber
        FROM dbo.FactInternetSales GROUP BY 1, 2 HAVING COUNT(*) > 1);

-- Une commande, un seul client.
SELECT COUNT(*) AS commandes_a_plusieurs_clients
FROM   (SELECT SalesOrderNumber
        FROM dbo.FactInternetSales
        GROUP BY 1 HAVING COUNT(DISTINCT CustomerKey) > 1);
```


© 2026 iAlexMG. Tous droits réservés.

Le second est le plus susceptible de trouver quelque chose sur une vraie source, et c'est le point de la question. Le premier vérifie une contrainte que le système source fait généralement respecter lui-même. Le second vérifie une hypothèse de cohérence que rien ne garantit dans une chaîne d'alimentation, où une commande peut être réaffectée, fusionnée ou corrigée à moitié.

Théorie : `Theorie_12_SQL.md`, section 4.

P4. `[OLAP]` Trois traitements.

Les écarter du rapport, en les documentant dans le rapport de contrôle. Simple, et il fait silencieusement baisser le chiffre d'affaires publié, ce qui est inacceptable sans le dire.

Les garder avec une marge `NULL` et une mesure de marge qui ignore les `NULL`. Le chiffre d'affaires reste juste, la marge est calculée sur une base partielle, et il faut publier la couverture, c'est-à- dire la part du chiffre d'affaires pour laquelle la marge est connue.

Les garder en retombant sur la colonne du fait, `TotalProductCost`, quand l'as-of join ne trouve rien. Le rapport est complet et repose sur deux définitions de la marge selon les lignes.

Le deuxième est le meilleur choix par défaut, à condition de publier la couverture : il ne cache rien et ne mélange pas deux définitions. Le troisième est défendable si la couverture est très faible et si l'on documente la règle de repli.

Théorie : `Theorie_12_SQL.md`, section 4.

P5. `[OLAP]` (avancé)

```sql
CREATE OR REPLACE TABLE RAPPORT.JOURNAL_CHARGEMENT (
    execution_id  VARCHAR,       -- une valeur par execution complete
    etape         VARCHAR,
    jour_traite   DATE,
    debut         TIMESTAMP,
    fin           TIMESTAMP,
    lignes        BIGINT,
    statut        VARCHAR,       -- OK, ECHEC, EN COURS
    message       VARCHAR
);
```

C'est la table du laboratoire, sous-partie 8.4, enrichie de quatre colonnes : l'identifiant d'exécution, qui relie entre elles les étapes d'un même lot, la journée traitée, le statut et le message. Le labo se contentait du minimum pour montrer le motif ; une table de journal qui sert vraiment porte ces quatre-là.

C'est le chargement lui-même qui l'alimente, à l'ouverture et à la fermeture de chaque étape, dans la même transaction que l'étape qu'il décrit. Écrit par un outil de surveillance extérieur, le journal mentirait le jour où le chargement s'arrête brutalement.

```sql
-- Ce que l'operateur lance le matin.
SELECT execution_id,
```


© 2026 iAlexMG. Tous droits réservés.

```
        MIN(debut)                                  AS debut,
        MAX(fin)                                    AS fin,
        COUNT(*)                                    AS etapes,
        COUNT(*) FILTER (WHERE statut = 'OK')       AS etapes_ok,
        COUNT(*) FILTER (WHERE statut <> 'OK')      AS etapes_en_echec,
        SUM(lignes)                                 AS lignes_traitees
FROM    RAPPORT.JOURNAL_CHARGEMENT
WHERE   debut >= CURRENT_TIMESTAMP - INTERVAL 1 DAY
GROUP   BY execution_id
ORDER   BY debut DESC;
```

Le `FILTER` est celui du Cours 06, qui passe sur nos deux terrains ; d'autres moteurs écrivent `COUNT_IF`, dont Snowflake d'après sa documentation, et la forme la plus portable des trois reste `SUM(CASE WHEN … THEN 1 ELSE 0 END)`.

Le point qui distingue une bonne réponse : l'absence de ligne est aussi une information. Un chargement qui n'a rien écrit dans le journal n'a pas réussi, il n'a pas démarré, et la requête du matin doit rendre ce cas visible plutôt que de renvoyer un tableau vide qu'on interprète mal.

Théorie : `Theorie_12_SQL.md`, section 4. Théorie : `Theorie_02_SQL.md`, section 6.

P6. `[OLAP]` (avancé)

Ce qui change. La partie 1 doit décider si les deux canaux se cumulent dans une même mesure ou se comparent, ce qui est une décision de cadrage et non de SQL. La partie 2 gagne une dimension, le canal, donc l'axe devient le produit des jours et des canaux si l'on veut une série par canal sans trou. La partie 4 double ses contrôles, puisque la seconde table de faits a ses propres clés étrangères, et gagne un contrôle nouveau : un même numéro de commande ne doit pas exister dans les deux tables. La partie 6 gagne le canal comme niveau de regroupement. La partie 8 gagne une étape, donc une ligne de journal.

Ce qui ne change pas. Les mesures de la partie 3, qui s'appliquent à toute série quotidienne. L'as-of join de la partie 5, qui porte sur la dimension produit et non sur le fait. Et la méthode elle-même.

Le piège à nommer, et c'est le cœur de la question : réunir deux tables de faits par `UNION ALL` suppose qu'elles aient exactement le même grain et les mêmes définitions de mesure. Si l'une porte les remises et l'autre non, ou si l'une est au grain article et l'autre au grain commande, la réunion produit un total plausible et faux.

Théorie : `Theorie_12_SQL.md`, sections 3 et 4.

Diagnostics

E1. Deux défauts.

Le premier est le grain : `COUNT(*)` compte des articles, pas des commandes. Il faut `COUNT(DISTINCT SalesOrderNumber)`.

© 2026 iAlexMG. Tous droits réservés.

Le second est la sargabilité : `YEAR(OrderDate)` applique une fonction à la colonne filtrée, ce qui empêche DuckDB d'écarter un groupe de lignes sur son minimum et son maximum sans le lire. Sur les 60 398 ventes, rangées dans un seul groupe, l'écart ne se voit pas encore ; le laboratoire le mesure à sa sous-partie 7.3, sur vingt millions de lignes. Il faut un intervalle semi-ouvert.

```
SELECT COUNT(DISTINCT SalesOrderNumber) FROM dbo.FactInternetSales
WHERE  OrderDate >= '2013-01-01' AND OrderDate < '2014-01-01';
```

Le premier défaut rend le chiffre faux, le second le rendra lent quand la table grossira. Une bonne réponse dit lequel des deux est le plus grave, et pourquoi : un chiffre lent finit par arriver, un chiffre faux est publié.

Théorie : `Theorie_12_SQL.md`, section 3.

E2. Les deux mois extrêmes sont tronqués. Décembre 2010 ne compte que trois jours de vente et janvier 2014 s'arrête au 28 : la chute et le creux sont des artefacts de bornage, pas des faits.

Dans la requête, on filtre sur une période à mois complets. Dans le livrable, on ajoute deux choses : la période retenue, écrite dans la note de cadrage, et une indication visible du dernier mois complet, faute de quoi le problème reviendra chaque fois que le rapport sera rejoué en cours de mois.

```
SELECT DATE_TRUNC('month', OrderDate) AS mois, SUM(SalesAmount) AS ventes
FROM   dbo.FactInternetSales
WHERE  OrderDate >= '2011-01-01' AND OrderDate < '2014-01-01'
GROUP  BY 1 ORDER BY 1;
```

Théorie : `Theorie_12_SQL.md`, section 4.

E3. Le groupement porte sur le nom du produit, donc sur une colonne de `dbo.DimProduct`, mais la jointure se fait sur `ProductKey`, qui désigne une version. Un produit à plusieurs versions apparaît autant de fois qu'il a de versions vendues, et son chiffre d'affaires est réparti entre elles.

```
SELECT p.ProductAlternateKey AS code_produit,
       ANY_VALUE(p.EnglishProductName) AS produit,
       SUM(f.SalesAmount) AS ventes
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimProduct AS p ON p.ProductKey = f.ProductKey
GROUP  BY 1 ORDER BY ventes DESC LIMIT 10;
```

On groupe sur la clé métier. Le nom peut lui aussi varier d'une version à l'autre, d'où `ANY_VALUE` plutôt qu'un groupement supplémentaire sur le nom, qui ramènerait le problème.

Théorie : `Theorie_12_SQL.md`, section 4.

E4. Deux causes, et ce sont celles du Cours 09. Les 200 versions historiques ont une `EndDate` antérieure à leur `StartDate`, donc leur intervalle est vide et n'apparie personne. Les 406 versions courantes ont `EndDate` à `NULL`, et une comparaison avec `NULL` ne vaut ni vrai ni faux, donc elles n'apparient personne non plus. Aucune ligne ne survit au `WHERE`, `SUM` d'un ensemble vide vaut `NULL`.

© 2026 iAlexMG. Tous droits réservés.

```sql
SELECT SUM(f.SalesAmount - v.StandardCost * f.OrderQuantity) AS marge
FROM   dbo.FactInternetSales AS f
JOIN   dbo.DimProduct AS ref ON ref.ProductKey = f.ProductKey
LEFT   JOIN LATERAL (
         SELECT p.StandardCost
         FROM   dbo.DimProduct AS p
         WHERE  p.ProductAlternateKey = ref.ProductAlternateKey
           AND  p.StartDate <= f.OrderDate
         ORDER  BY p.StartDate DESC
         LIMIT  1
       ) AS v ON TRUE;
```

Une bonne réponse ajoute que `SUM` sur zéro ligne renvoyant `NULL` plutôt que zéro est ici une chance : un zéro se serait fondu dans un tableau de bord, alors qu'un `NULL` se remarque.

Théorie : `Theorie_12_SQL.md`, section 4.

E5. Le `ROLLUP` à deux niveaux produit trois familles de lignes : le détail par catégorie et par pays, un sous-total par catégorie, et le total général. La somme de la colonne vaut donc trois fois le chiffre d'affaires, puisque chaque montant y figure trois fois, une fois dans le détail, une fois dans son sous-total, une fois dans le total.

Ce qui manque au livrable est ce qui permettrait de distinguer les trois familles, c'est-à-dire les colonnes issues de `GROUPING()`. Le `COALESCE` qui écrit « Total » aggrave le problème, parce qu'il donne l'apparence d'une étiquette de données à une ligne d'agrégat, et parce qu'une vraie valeur manquante porterait la même étiquette.

```sql
SELECT COALESCE(categorie, 'Total') AS categorie,
       COALESCE(pays, 'Total')      AS pays,
       GROUPING(categorie)          AS est_total_categorie,
       GROUPING(pays)               AS est_total_pays,
       SUM(ventes)                  AS ventes
FROM   ventes_detail
GROUP  BY ROLLUP (categorie, pays);
```

Est également acceptable une réponse qui livre le détail seul et laisse l'outil de restitution calculer ses sous-totaux, ce qui est souvent le meilleur choix quand le destinataire travaille dans un tableur.

Théorie : `Theorie_12_SQL.md`, section 4.

E6. Le `DELETE` porte sur un jour, l'`INSERT` sur tous les jours à partir de ce jour. Le second passage réinsère donc toutes les journées postérieures, qui n'ont pas été supprimées.

```sql
DELETE FROM rapport.fait WHERE jour = :jour;
INSERT INTO rapport.fait SELECT * FROM source.fait WHERE jour = :jour;
```


© 2026 iAlexMG. Tous droits réservés.

La règle à retenir est que le filtre du `DELETE` et celui de l'`INSERT` doivent désigner exactement la même tranche, et c'est la seule chose à vérifier en relisant ce genre de code.

Le contrôle de la partie 4 qui l'aurait détecté est celui de l'unicité de la clé de ligne : un doublon de `SalesOrderNumber` et `SalesOrderLineNumber` serait apparu dès le premier rejeu. Une bonne réponse ajoute que ce contrôle aurait dû tourner après le chargement et faire échouer le traitement, pas seulement figurer dans un tableau que personne ne regarde.

Théorie : `Theorie_12_SQL.md`, section 4.

Questions à développer

A1. Une bonne réponse s'appuie sur ce qui, dans le cadrage, ne se retrouve nulle part dans le code. Le SQL dit comment on calcule, jamais pourquoi on a choisi ce calcul-là. La décision de faire valoir zéro un jour sans vente, celle de borner la période aux mois complets, celle de compter les commandes plutôt que les articles : chacune est un choix parmi plusieurs, et aucune ne laisse de trace lisible dans la requête. Faite avant, la décision est prise une fois et documentée. Faite après, elle est reconstituée par déduction, par quelqu'un qui ne sait pas s'il lit une intention ou un accident.

Concrètement, sur ce projet, le cadrage tardif se paie sur la moyenne quotidienne. Un rapport livré sans avoir décidé ce que vaut un jour sans vente affichera une moyenne calculée sur les jours actifs, parce que c'est ce qu'un `GROUP BY` produit naturellement. Trois mois plus tard, quelqu'un reconstruira l'axe et obtiendra un chiffre inférieur, les deux services se disputeront, et personne ne saura lequel des deux répond à la question posée à l'origine, parce que la question n'a jamais été écrite.

Théorie : `Theorie_12_SQL.md`, section 2.

A2. L'ordre du laboratoire est pédagogique : il fait construire des mesures d'abord pour que les contrôles portent sur quelque chose de tangible, et pour que l'apprenant voie ce que les contrôles protègent. Ce n'est pas l'ordre d'un projet réel.

Ce qu'il coûte est réel aussi : on peut passer une heure à peaufiner une moyenne mobile sur une donnée dont on apprendra ensuite qu'elle a un défaut qui change tout. Sur ce projet, c'est arrivé : les intervalles de validité cassés, découverts en partie 4, décident de la façon d'écrire la partie 5.

L'ordre à adopter sur un projet réel est cadrer, puis contrôler, puis construire. On écrit d'abord les anti-jointures de référence, les contrôles de grain et les contrôles de cohérence, en une demi-heure, avant toute mesure. Une bonne réponse nuance : ces contrôles ne sont pas tous imaginables à l'avance, et une deuxième passe de contrôle après la construction reste utile, parce que construire une mesure révèle des questions que le schéma seul ne posait pas.

Théorie : `Theorie_12_SQL.md`, sections 2 et 4.

A3. Les trois traitements sont de faire confiance au fait, de recalculer, ou de publier les deux.

Le choix par défaut raisonnable est de recalculer et de publier la couverture, parce que le recalcul est vérifiable et que la colonne du fait ne l'est pas. Mais ce choix se renverse dans deux cas.

© 2026 iAlexMG. Tous droits réservés.

Si la marge n'est pas une mesure principale du rapport, la complexité du recalcul ne se justifie pas, et faire confiance au fait en documentant l'écart est la bonne économie de moyens.

Si l'écart est important et explicable par une différence de définition, par exemple parce que le fait inclut un coût de transport que la dimension ignore, alors c'est le fait qui a raison et le recalcul qui est faux. Une bonne réponse insiste sur ce point : un écart n'est pas une erreur tant qu'on n'a pas identifié laquelle des deux mesures répond à la question.

Le cas de l'écart important et inexplicable à la veille de la livraison se traite en ne livrant pas la mesure. On livre le reste du rapport, on retire la marge, on écrit ce qu'on a constaté et ce qu'on cherche. Livrer un chiffre qu'on ne sait pas expliquer est le seul choix qui soit défendable en aucune circonstance, parce qu'il transfère le risque sur quelqu'un qui n'a pas l'information pour l'évaluer.

Théorie : `Theorie_12_SQL.md`, section 4.

A4. L'objection a du fond, et il faut commencer par là. Un analyste n'est pas administrateur de base de données, la frontière existe, et beaucoup d'organisations ont une équipe dont c'est le travail. Se préoccuper d'idempotence quand on est seul à lancer son script à la main, sur une table que personne d'autre ne lit, est effectivement du zèle.

Le scénario qui renverse l'objection est celui de l'absence. Votre rapport tourne chaque nuit. Vous partez trois semaines. La nuit du mardi, le chargement échoue à mi-parcours pour une raison qui n'a rien à voir avec lui. Quelqu'un le relance mercredi matin, sans savoir ce qui a déjà été écrit. Si le chargement n'est pas idempotent, il double des lignes, et le tableau de bord de la direction affiche un chiffre d'affaires en hausse de quarante pour cent. Personne ne le remarque avant la réunion mensuelle, et quand on le remarque, personne ne sait quelles journées sont doublées.

La conclusion à tirer est que l'idempotence n'est pas une exigence technique, c'est ce qui rend votre travail exploitable par quelqu'un d'autre que vous. Un rapport juste que vous seul savez relancer est un rapport qui a une date d'expiration, et cette date est celle de votre départ.

Théorie : `Theorie_12_SQL.md`, sections 4 et 5.

A5. Le Cours 08 n'est pas remobilisé parce que l'entrepôt d'Adventure Works ne contient aucune donnée semi-structurée, et parce que fabriquer du JSON pour l'occasion irait contre la leçon de ce module, qui est justement de ne pas en mettre là où il n'y a pas lieu. Un capstone qui inventerait le besoin enseignerait le contraire de ce qu'il prétend.

Si la source en contenait, l'étape correspondante viendrait juste avant l'étape de contrôle : on aplatit les tableaux, on type les valeurs extraites, puis on contrôle. L'ordre importe, parce que contrôler un document non aplati revient à contrôler une chaîne de caractères.

Deux conséquences suivraient. À l'étape de contrôle, il faudrait ajouter les contrôles propres au semi-structuré : le compte des valeurs tombées à `NULL` après conversion tolérante, et la vérification que le nombre de lignes après aplatissement égale la somme des tailles de tableau. À l'étape de mesure du coût, l'extraction deviendrait probablement le poste le plus cher, et la parade serait la matérialisation des chemins fréquents en colonnes, qui est le motif de la couche brute et de la couche exploitée.

Théorie : `Theorie_12_SQL.md`, section 6.

© 2026 iAlexMG. Tous droits réservés.

Mises en situation

M1. Réponse défendable.

La démarche commence par le tableau existant, et non par le nouveau. On le lit, on le fait tourner, on note ce qu'il calcule réellement, et on écrit cette description en français. Cette rétro-ingénierie n'est pas du temps perdu : c'est elle qui donnera les définitions que personne ne sait énoncer, et c'est aussi elle qui expliquera la divergence entre les deux services.

Ce qu'on demande, et à qui. Aux deux services, la requête ou l'export exact qu'ils citent, pas leur chiffre. Dans neuf cas sur dix, la divergence vient d'un filtre différent, d'une période décalée d'un jour, ou d'un grain confondu, et la comparaison des deux extractions le montre en une heure. À la direction, ce qu'elle veut décider avec ce tableau de bord, parce que c'est la seule question qui permette de trancher entre deux définitions également valides.

La divergence entre les services se traite en la reformulant. On ne cherche pas qui a raison, on cherche quelle question chacun a posée. Presque toujours, les deux ont raison sur des questions différentes, et le livrable devra soit trancher, soit publier les deux mesures avec leurs noms distincts. Présenté ainsi, personne ne perd la face, et la conversation porte sur les définitions plutôt que sur les personnes.

Ce qu'on livre en premier n'est pas un tableau de bord, c'est la note de cadrage : les définitions retenues, la période, le grain, et la liste des divergences avec l'ancien tableau, chiffrées. Une bonne réponse dit pourquoi : c'est le seul livrable qui permette à la direction de valider avant qu'on ait construit quoi que ce soit, et c'est aussi celui qui protège du prochain « comme l'an dernier ».

Théorie : `Theorie_12_SQL.md`, section 2.

M2. Réponse défendable.

Les causes, de la plus probable à la moins probable.

La source a été corrigée. Un chargement amont a rejoué octobre avec des données complétées, ce qui est le cas le plus fréquent et le plus banal. On le vérifie en comparant le compte de lignes et la date de dernière modification de la source pour octobre.

Le chargement a été relancé et a doublé des lignes. On le vérifie par le contrôle d'unicité de la clé, et par le journal de chargement s'il existe.

Une dimension a évolué, et un rattachement a changé. Un produit passé d'une sous-catégorie à une autre déplace du chiffre d'affaires entre catégories, et un rapport filtré sur une catégorie change sans que le total bouge. On le vérifie en comparant le total général : s'il n'a pas bougé, c'est une redistribution.

Le rapport lit une vue dont la définition a changé en amont. Rare, et facile à écarter en regardant l'historique des modifications d'objets.

Enfin, le directeur s'est trompé de ligne ou de version. C'est la cause la moins probable à énoncer et la plus fréquente en réalité, et il faut se garder de commencer par elle.

Ce qu'on répond au directeur avant d'avoir fini : qu'on a bien reproduit son constat, que le chiffre a effectivement changé, qu'on connaît les quatre familles de causes possibles, et qu'on répond avant telle heure. Ce qu'il ne faut pas faire est de répondre « je regarde » sans donner d'échéance, ni avancer une hypothèse avant de l'avoir vérifiée.

© 2026 iAlexMG. Tous droits réservés.

Ce qu'on met en place pour la suite : un instantané daté des mesures publiées, conservé à chaque exécution, qui permet de répondre à « qu'affichait le rapport tel jour » sans dépendre de la mémoire de personne. Une bonne réponse ajoute une note dans le tableau de bord lui-même indiquant que les chiffres des trois derniers mois sont susceptibles d'être révisés, ce qui change la nature de la conversation.

Théorie : `Theorie_12_SQL.md`, sections 4 et 5.

M3. Réponse défendable, en deux minutes.

On commence par reconnaître ce que la question a de légitime : la colonne existe, elle est généralement juste, et s'en servir est le choix par défaut raisonnable. Personne ne devrait recalculer par principe ce qu'un entrepôt a déjà calculé.

On enchaîne sur la raison du recalcul, qui n'est pas la méfiance mais la vérification. La colonne du fait porte le coût tel qu'il était au moment du chargement, et rien dans le modèle ne garantit que ce soit le coût du moment de la vente. Le recalcul est donc d'abord un contrôle : il compare deux définitions et mesure leur écart.

On termine sur ce qu'on a mesuré, et c'est ce que le recruteur attend. Sur ce jeu de données, la comparaison a montré que la clé du fait et la version désignée par les dates de la dimension diffèrent sur une ligne sur dix, et que 200 intervalles de validité sur 200 sont incohérents. Autrement dit, le contrôle a trouvé quelque chose, ce qui justifie de l'avoir écrit.

À la seconde question, sur les quarante minutes, on ne se défend pas, on requalifie. Une écriture latérale s'exécute une fois par ligne de fait, donc son coût croît avec le volume, et sur un gros entrepôt on la remplace par la jointure sur inégalité avec `QUALIFY`, qui laisse le moteur travailler par blocs. On ajoute deux choses : que la version portable reste la bonne réponse en entretien parce qu'elle montre le mécanisme, et que sur un entrepôt de production le contrôle n'a pas besoin de tourner à chaque exécution du rapport, puisqu'il peut tourner une fois par semaine sur un échantillon. Une bonne réponse finit par la question qu'on retourne : à quelle fréquence ce contrôle a-t-il besoin de tourner chez vous.

Théorie : `Theorie_12_SQL.md`, section 5.

M4. Réponse défendable.

Ce qu'on fait en premier n'est ni de réécrire ni d'ajouter les deux mesures : c'est de construire un harnais de non-régression. Concrètement, on exécute le rapport existant sur plusieurs mois, on enregistre ses sorties complètes dans des tables datées, et on écrit la requête de comparaison. Sans ce harnais, toute modification est un pari, et c'est précisément pour cela que personne n'ose y toucher depuis des années.

Le plan, en étapes.

La première semaine, le harnais et la lecture. On enregistre les sorties de référence, et on lit les 900 lignes en les annotant, sans rien changer. On produit une description en français de ce que le rapport calcule, section par section.

La deuxième semaine, le découpage. On casse la requête unique en couches nommées, chacune une vue, en vérifiant après chaque découpe que le harnais est au vert. Aucune logique ne change, seule la forme. C'est l'étape qui rend tout le reste possible, et c'est celle qu'on est tenté de sauter.

© 2026 iAlexMG. Tous droits réservés.

La troisième semaine, les deux mesures demandées, qui deviennent faciles parce qu'elles se posent sur une couche identifiée. Et la mesure du coût, qui devient possible parce que chaque couche se chronomètre séparément.

La non-régression se prouve en comparant les sorties ligne à ligne et colonne à colonne, sur plusieurs périodes, avec une tolérance nulle. On compare aussi les comptes de lignes, parce qu'une ligne manquante ne se voit pas dans une comparaison de totaux.

Le cas où les chiffres diffèrent et où les vôtres semblent plus justes est le plus délicat, et c'est le cœur de la question. On ne livre pas la correction dans la même modification que la refonte, sinon personne ne saura ce qui a changé les chiffres. On isole le défaut, on le documente, on chiffre son impact, et on le porte à la direction comme une décision qui lui appartient : corriger l'historique, corriger à partir d'une date, ou continuer avec le calcul actuel en le documentant. Une bonne réponse insiste sur ce dernier point : découvrir qu'un rapport de direction est faux depuis trois ans n'est pas une décision technique.

Théorie : `Theorie_12_SQL.md`, sections 2 et 5.

© 2026 iAlexMG. Tous droits réservés.